Rush Hour Puzzle

Elementos de Inteligência Artificial e Ciência de Dados

abril.2024

Beatriz Pereira 202304769
Carolina Leite 202307856
Inês Santos 202305589

Rush Hour Puzzle

In the Rush Hour game, the goal is to remove the red car from the board. For that, the player needs to move the other vehicles to clear the path.
The game has the following rules:

- 3 levels of difficulty: 6x6 board (beginner, intermediate, advanced, expert), 9x9 board, 12x12 board;
- 16 vehicles: 12 cars and 4 trucks;
- Each car occupies 2 squares while a truck ocupies 3 squares;
- Rottion is not allowed, only moving the vehicles vertically and horizontally

We will try to solve it by implementing 3 different algorithms:

- Depth-first search, Breadth-first search, A* Algorithm.

Formulation of the problem as a search problem

State representation

Represented by the board and the vehicles on the board.
board → dimension
vehicles → identification (id), position (x, y), orientation (orientation), size (length)

Operators

Consist on moving the blocks in any of the four cardinal directions (up, down, left or right), provided there is no obstacle in the way, each valid move has a cost of 1.

Inicial state

It is defined by the initial positions of all the vehicles on the board.

Objetive test

Is to move the red car to the exit position;
Checks if the red car is there.

Names	Preconditions	Effects	Cost
move_horizontal_left	vehicle.orientation == ‘ H ’ and vehicle.x != 0 and self.board[vehicle.y][vehicle.x – 1] == ‘ ’	vehicle.x – 1	1
move_horizontal_right	vehicle.orientation == ‘ H ’ and vehicle.x + vehicle.length - 1 != self.width - 1 and self.board[vehicle.y][vehicle.x + vehicle.length] == ‘ ’	vehicle.x + 1	1
move_vertical_up	vehicle.orientation == ‘V’ and vehicle.y != 0 and self.board[vehicle.y – 1][vehicle.x] == ‘ ’	vehicle.y - 1	1
move_vertical_down	vehicle.orientation == ‘V’ and self.board[vehicle.y + vehicle.length][vehicle.x]== ‘ ’	vehicle.y +1	1

Formulation of the problem as a search problem

Heuristic/evaluation functions

Manhattan distance

The Manhattan distance heuristic, estimates the shortest distance between the red car and the possible exit leading to a solution more quickly. For the Rush Hour game it can be defined as follows:

- Recognize the red car on the board as the focal point of the heuristic;
- For each coordinate occupied by the red car, it calculates the distance from Manhattan to the exit position, a distance that represents the minimum number of moves necessary to reach the exit, assuming that the board is empty;
- After obtaining the distances for all the coordinates occupied by the red car choose the maximum distance, because choosing the furthest coordinate allows us to know that we need at most that number of movements to reach the exit position;
- The heuristic value is used as a guide for the search algorithm to explore the state space more effectively, prioritizing the states where the red car is closest to the exit position.

Implementation work already carried out

Development Environment:

Programming Language: Python
IDE: Visual Studio Code

Algorithms:

Breath-first serch
Depth-first search
A* Algorithm

Data Structures:

To represent the board and vehicle positions, we use a matrix or a grid.
To represent the state of the puzzle, we use lists or tuples.

File Structure:

Organization of Python modules and configuration files:
Main Module: for game logic and search algorithms.
GUI Module: for graphical user interface implementation using Pygame.
Utilities Module: contains utility or helper functions that are used in the Project.

Text/Graphical Interface:

Pygame

Algorithms implemented and experimental results

Board 6x6 Level	Algorithm	Time	Nodes	Length solution
beginner	BFS	0.6562466621398926	1072	17
beginner	DFS	0.046881914138793945	163	160
beginner	A*	1.7731499671936035	4	
intermediate	BFS	0.3437495231628418	849	57
intermediate	DFS	0.09393453598022461	327	161
intermediate	A*	1.0236270427703857	11	
advanced	BFS	0.06249833106994629	262	50
advanced	DFS	0.031258583068847656	132	108
advanced	A*	0.24954986572265625	3	
expert	BFS	1.2190465927124023	3927	70
expert	DFS	0.6250030994415283	1375	1117
expert	A*	5.690542221069336	72	

Algorithms implemented and experimental results

Board	Algorithm	Time	Nodes	Length solution
game_9x9	BFS	421.68164134025574	347037	52
game_9x9	DFS	52.863196849823	32719	32719
game_9x9	A*	973.6412715911865	39424	

In the game, there are three diferentes search methods:

BFS (breadth-first search); DFS (depth-first search); A* algorithm.

Through experimental results, we couldn't finish A* algorithm as we intended to.

The best algorithm is BFS (breadth-first search).

Note: The board game_12x12 was not possible to complete as it required too many computacional resources to present results. After more than 3 hours of running, we decided to end it, without obtaining the desired results.

BOARD	LEVEL	ALGORITHM BFS
game_ 6x6	beginner	['G to the right', 'P up', 'B to the left', 'Y down', 'O up', 'W to the left', 'B to the left', 'Y down', 'W to the left', 'B to the left', 'V down', 'Y down', 'V down', 'R to the right', 'R to the right', 'R to the right']
game_ 6x6	intermediate	['Y down', 'Y down', 'Y down', 'R to the left', 'O up', 'O up', 'B to the left', 'V up', 'W to the right', 'V up', 'V up', 'B to the right', 'O down', 'O down', 'R to the right', 'Y up', 'O down', 'B to the left', 'Y up', 'B to the left', 'Y up', 'B to the left', 'P down', 'G to the right', 'G to the right', 'G to the right', 'P up', 'B to the right', 'Y down', 'B to the right', 'Y down', 'B to the right', 'O up', 'W to the left', 'Y down', 'R to the left', 'O up', 'W to the left', 'O up', 'B to the left', 'V down', 'O up', 'R to the right', 'Y up', 'W to the left', 'B to the left', 'V down', 'Y up', 'Y up', 'B to the left', 'P down', 'P down', 'P down', 'R to the right', 'R to the right', 'R to the right']
game_ 6x6	advanced	['P down', 'B to the right', 'P down', 'R to the left', 'O down', 'G to the right', 'B to the right', 'O down', 'O down', 'R to the right', 'P up', 'O down', 'B to the left', 'P up', 'B to the left', 'P up', 'B to the left', 'Y down', 'G to the right', 'G to the right', 'G to the right', 'Y up', 'B to the right', 'P down', 'B to the right', 'P down', 'B to the right', 'O up', 'W to the left', 'P down', 'R to the left', 'O up', 'W to the left', 'O up', 'B to the left', 'O up', 'R to the right', 'P up', 'W to the left', 'B to the left', 'P up', 'P up', 'B to the left', 'Y down', 'Y down', 'Y down', 'R to the right', 'R to the right', 'R to the right']
game_ 6x6	expert	['G to the right', 'V up', 'V up', 'R to the left', 'B up', 'B up', 'S to the right', 'S to the right', 'S to the right', 'B down', 'B down', 'R to the right', 'V down', 'G to the left', 'V down', 'V down', 'R to the left', 'B up', 'V down', 'B up', 'B up', 'C to the left', 'C to the left', 'O down', 'M to the left', 'P up', 'S to the right', 'C to the left', 'B down', 'O down', 'B down', 'B down', 'R to the right', 'R to the right', 'R to the right', 'B up', 'B up', 'B up', 'C to the right', 'V up', 'V up', 'V up', 'C to the left', 'B down', 'G to the right', 'V up', 'B down', 'B down', 'R to the left', 'R to the left', 'O up', 'S to the left', 'P down', 'R to the left', 'B up', 'B up', 'S to the left', 'S to the left', 'O down', 'S to the left', 'B down', 'B down', 'R to the right', 'R to the right', 'R to the right', 'B up', 'W to the left', 'P down', 'R to the right']
game_ 9x9		['V to the right', 'P up', 'B to the left', 'G to the left', 'W down', 'M to the right', 'F to the left', 'T down', 'N to the left', 'K up', 'V to the right', 'F to the left', 'F to the left', 'L down', 'L down', 'J to the left', 'I up', 'L down', 'I up', 'L down', 'F to the right', 'H to the right', 'E up', 'L down', 'F to the right', 'H to the right', 'F to the right', 'H to the right', 'F to the right', 'H to the right', 'K down', 'N to the right', 'T up', 'E up', 'R to the left', 'C up', 'C up', 'A to the right', 'O up', 'Y to the left', 'S down', 'D down', 'C up', 'C up', 'R to the right', 'R to the right', 'R to the right', 'R to the right', 'R to the right', 'R to the right', 'R to the right']

Bibliografic search

- https://blog.devgenius.io/mastering-the-rush-hour-puzzle-how-bfs-can-help-you-find-the-shortest-solution-f2107eb6f19
- https://ursinus-cs477-f2021.github.io/CoursePage/Assignments/HW2_RushHour/
- https://chat.openai.com
- https://ursinus-cs477-f2021.github.io/CoursePage/index.html
- https://github.com/KaKariki02/rushHour